

MIAE 212: Computer Programming for Engineers

Assignment 1 & Course Exercises: Fully Solved & Annotated Master Guide

Question 1: Mini-Course Lesson 4 Exercises & Code Tracing

Part (a): Lesson 4 Arithmetic & Expression Tracing

Analyze the following code fragment line by line and determine the exact screen output without running the code:

```
int q;
double x, y, z = 0;
double d, r1 = 1.234567890123456, r2 = 1.234567890123455;

q = 7 - (7 / 3) * 3;
x = 1 / 3 * 10.0;
y = -1.0 / z;
d = r1 - r2;

cout << q << x << "\n" << y << "\n" << d << "\n";
```

Line-by-Line Execution Analysis:

1. `q = 7 - (7 / 3) * 3;`
 - `7 / 3` is integer division. $7/3 = 2.3333\dots$, which truncates to `2`.
 - `2 * 3 = 6`.
 - `7 - 6 = 1`. Thus, `q = 1`.
2. `x = 1 / 3 * 10.0;`
 - Operator precedence evaluates left to right: `1 / 3` occurs first.
 - `1 / 3` is integer division, which truncates to `0`!
 - `0 * 10.0 = 0.0`. Thus, `x = 0`.
3. `y = -1.0 / z;`
 - Floating-point division by zero with a negative numerator (`-1.0 / 0.0`).
 - In standard IEEE 754 floating-point arithmetic, this does not crash; it evaluates to `-inf`.
4. `d = r1 - r2;`
 - Both `r1` and `r2` are 64-bit `double` variables with 16 significant decimal figures.
 - `r1 - r2 = 1.234567890123456 - 1.234567890123455 = 1.0e-15`. Thus, `d = 1.0e-15`.

Console Output:

```
10
-inf
1e-15
```

(Note: `cout << q << x` prints `1` immediately followed by `0` with no whitespace, displaying `10` !)

Part (b): Lesson 4 Precision Limits

Consider:

```
r3 = 1.0 - (1.0 - 1.0e-15);
r4 = 1.0 - (1.0 - 1.0e-20);
```

- **r3**: Because 1.0×10^{-15} is above the double-precision machine epsilon ($\epsilon \approx 2.22 \times 10^{-16}$), the arithmetic evaluates accurately to **1.0e-15**.
- **r4**: Because 1.0×10^{-20} is significantly smaller than machine epsilon, $1.0 - 1.0 \times 10^{-20}$ rounds down to exactly 1.0. Thus $1.0 - 1.0 = \mathbf{0.0}$.

Question 2: Memory Limits, `sizeof` & Overflow Tracing

Problem Statement

Given that 3.4×10^{38} is the maximum positive `float` value, determine the exact output of the following program on a PC:

```
int q;
float x, y, r = 5, v;
float r1 = 1.234567890123456, r2 = 1.234567890123455;
double w = 1.0e19, d;
double u1 = 1.234567890123456, u2 = 1.234567890123455;

d = r1 - r2;
q = sizeof(float) * sizeof(int) - sizeof(double) * sizeof(char);
x = 4 / 3 * 3.14159 * r * r;
y = 3.5 * w * w;
y = y * 0.5;
x = 7.7;
x += sin(x);
v = u1 - u2;

cout << d << "\n" << q << "\n" << x << "\n" << y << "\n" << v;
```

In-Depth Mathematical & Architecture Trace

1. **d = r1 - r2;**
 - Variables **r1** and **r2** are declared as `float` (single precision, ~7 significant decimal digits).
 - Both **1.234567890123456** and **1.234567890123455** differ only at the 16th decimal digit.
 - When stored in a 32-bit `float`, both values are rounded to the identical float representation: **1.2345679**.
 - Therefore, **r1 - r2 = 0.0f**. Storing this in double **d** yields **d = 0**.
2. **q = sizeof(float) * sizeof(int) - sizeof(double) * sizeof(char);**
 - On a PC: `sizeof(float) = 4`, `sizeof(int) = 4`, `sizeof(double) = 8`, `sizeof(char) = 1`.
 - $q = (4 \times 4) - (8 \times 1) = 16 - 8 = \mathbf{8}$.
3. **x = 4 / 3 * 3.14159 * r * r;**
 - $4 / 3$ is integer division, truncating to **1**!
 - $x = 1 \times 3.14159 \times 5 \times 5 = 1 \times 3.14159 \times 25 = 78.53975$.
 - (Note: Later reassigned to 7.7 below).

4. `y = 3.5 * w * w; then y = y * 0.5;`
- `w = 1.0e19`. In double precision: $w^2 = 1.0 \times 10^{38}$.
 - $3.5 \times 1.0 \times 10^{38} = 3.5 \times 10^{38}$.
 - However, `y` is a `float` whose maximum representable value is 3.4×10^{38} !
 - $3.5 \times 10^{38} > 3.4 \times 10^{38}$, causing a **floating-point overflow**.
 - In IEEE 754, `y` becomes `inf` (Positive Infinity).
 - Multiplying infinity by 0.5 (`y = y * 0.5`) leaves it as `inf`.
5. `x = 7.7; x += sin(x);`
- $\sin(7.7 \text{ radians}) = \sin(7.7 - 2\pi) = \sin(1.416815) \approx 0.988469$.
 - $x = 7.7 + 0.988469 = \mathbf{8.68847}$.
6. `v = u1 - u2;`
- `u1` and `u2` are `double` variables.
 - $u_1 - u_2 = 1.234567890123456 - 1.234567890123455 = 1.0 \times 10^{-15}$.
 - Stored in float `v`: 1.0×10^{-15} is well within the float range (10^{-38}).
 - Thus, `v = 1e-15`.

Program Output:

```
0
8
8.68847
inf
1e-15
```

Question 3 & 4: Control Flow, Floating Epsilon & Integer Overflow

Question 4 Problem Statement

Determine the exact console output of the following program:

```

int i, j;
double y = 2.000000000001, z;

z = 1.0 - cos(2 * 3.14159);
if (z == 0.0) {
    cout << endl << "z = 0.0";
}

if (abs(y - 2.0) < 1.0e-7) {
    cout << "\ny = 2.0";
}

if (sin(y) < 0.7) {
    cout << "\nsin(y) < 0.7";
} else {
    cout << "\nsin(y) >= 0.7";
    if (sin(y) > 0.95) {
        cout << "\nsin(y) > 0.95";
    } else {
        cout << "\n0.7 < sin(y) <= 0.95";
    }
}

i = 1;
for (j = -1; j <= 15; j++) {
    i *= 10;
    if (i < 0) break;
}
cout << "\n" << j;

if (j > 5) {
    cout << "\nterminating program";
    exit(0);
}
cout << "\nprogram complete";

```

Step-by-Step Execution Trace

1. Test `if (z == 0.0)`:

- $2 \times 3.14159 = 6.28318$.
- $\cos(6.28318) = 0.99999999997 \dots \neq 1.0$.
- $z = 1.0 - \cos(6.28318) \approx 2.7 \times 10^{-11} \neq 0.0$.
- The condition `z == 0.0` is **false**. Nothing is printed.

2. Test `if (abs(y - 2.0) < 1.0e-7)`:

- $y = 2.000000000001$.
- $|y - 2.0| = 1.0 \times 10^{-12}$.
- Since $1.0 \times 10^{-12} < 1.0 \times 10^{-7}$, condition is **true**!
- Prints: `\ny = 2.0`.

3. Test `if (sin(y) < 0.7)`:

- $\sin(2.000000000001 \text{ rad}) \approx 0.909297$.
- $0.909297 < 0.7$ is **false** $\implies$ branches to **else**.
- Prints: `\nsin(y) >= 0.7`.

- Nested test: `sin(y) > 0.95` ($0.909297 > 0.95$) is `false` $\implies$ branches to nested `else`.
- Prints: `\n0.7 < sin(y) <= 0.95`.

4. Loop Tracing with Integer Overflow:

- i starts at 1. In each iteration, i is multiplied by 10.
- Signed 32-bit `int` range is up to $+2,147,483,647$ (2.14×10^9).

Iteration	j value	i calculation	i value in memory	Condition $i < 0$
1	-1	1×10	10	False
2	0	10×10	100	False
3	1	100×10	1,000	False
4	2	$1,000 \times 10$	10,000	False
5	3	$10,000 \times 10$	100,000	False
6	4	$100,000 \times 10$	1,000,000	False
7	5	$1,000,000 \times 10$	10,000,000	False
8	6	$10,000,000 \times 10$	100,000,000	False
9	7	$100,000,000 \times 10$	1,000,000,000	False
10	8	$1,000,000,000 \times 10$	$10^{10} \implies$ OVERFLOW!	True ($i =$ 1410065408 or negative)
(Specifically, 10^{10} in binary wraps around to a negative two's complement value, triggering <code>break</code> when $j = 8$ or $j = 9$ depending on integer signed multiplication wrap).				

- On 32-bit x86/x64: $10^{10} \pmod{2^{32}} = 1,410,065,408 > 0$. On next step ($j = 9$), 1.41×10^{10} wraps to negative, breaking at $j = 9$.
- Prints: `\n9`.

5. Termination Test:

- `if (j > 5)`: Since $9 > 5$, condition is `true`!
- Prints: `\nterminating program`.
- Calls `exit(0)` $\implies$ immediate program termination (skips `program complete`).

Program Output:

```
y = 2.0
sin(y) >= 0.7
0.7 < sin(y) <= 0.95
9
terminating program
```

Question 5: Loop Target Interval Search

Problem Statement

Write a program that calculates $f = \sin(t)$ in a **for** loop for $t = 0, 0.1, 0.2, \dots, 1.0 \times 10^7$ and stops the loop when $0.9 < f < 0.93$ using just **one if statement**. Print out t and f when this occurs.

Complete C++ Implementation

```
#include <iostream>
#include <cmath>

using namespace std;

int main()
{
    double t = 0.0, f = 0.0;

    for (t = 0.0; t <= 1.0e7; t += 0.1) {
        f = sin(t);

        // Combined conditional using logical AND (&&)
        if (f > 0.9 && f < 0.93) {
            break; // Stop loop immediately upon condition
        }
    }

    cout << "Target condition met:\n";
    cout << "t = " << t << "\n";
    cout << "f = " << f << "\n";

    return 0;
}
```

Execution Result:

- At $t = 1.1$ rad: $\sin(1.1) = 0.8912$ (too low).
- At $t = 1.2$ rad: $\sin(1.2) = 0.9320$ (slightly above 0.93).
- Cycle repeats in the next wave period ($t \approx 1.2 + 2\pi$):
 - At $t = 2.0$ rad: $\sin(2.0) = 0.909297$!
 - Since $0.9 < 0.909297 < 0.93$, the loop immediately breaks!
 - **Output:** $t = 2.0$, $f = 0.909297$.

Question 6: Successive Halving & Scientific Formatting

Problem Statement

Write a program that inputs a number x from the keyboard. It then divides x by 2 repeatedly until $x < 1.0 \times 10^{-7}$ and $\sin(1.0/x) > 0.9$. Print out final values of x and $\sin(1.0/x)$ in scientific notation with a precision of 10 digits.

Complete C++ Implementation

```
#include <iostream>
#include <cmath>

using namespace std;

int main()
{
    double x;
    cout << "Input starting value x: ";
    cin >> x;

    // Safety loop guard against infinite cycling
    for (int i = 0; i < 10000000; i++) {
        // Short-circuit check: ensures x is small AND sin(1/x) > 0.9
        if (x < 1.0e-7 && sin(1.0 / x) > 0.9) {
            break;
        }
        x /= 2.0; // Equivalent to x = x / 2.0;
    }

    // Set console output to scientific notation with 10 decimal digits
    cout << scientific;
    cout.precision(10);

    cout << "\nFinal Results:\n";
    cout << "x          = " << x << "\n";
    cout << "sin(1.0/x) = " << sin(1.0 / x) << "\n";

    return 0;
}
```

Question 7: Numerical Grid Search for Global Maximization

Problem Statement

Write a program that approximately determines the maximum value of the function:

$$f(x) = \sin(x^2 - 1.0) \cdot (x - 3.0) + \cos(x)$$

over the range $0 \leq x \leq 10\pi$. Evaluate the function at intervals of $dx = 0.01$ in a `for` loop. If $f > m$, set $m = f$. Set initial $m = f(0)$. Print the maximum value.

Complete C++ Implementation

```
#include <iostream>
#include <cmath>

using namespace std;

int main()
{
    const double PI = 4.0 * atan(1.0); // Exact PI
    const double x_start = 0.0;
    const double x_end = 10.0 * PI;
    const double dx = 0.01;

    // Initialize m to f(0)
    double m = sin(x_start * x_start - 1.0) * (x_start - 3.0) + cos(x_start);
    double x_at_max = x_start;

    for (double x = x_start; x <= x_end; x += dx) {
        double f = sin(x * x - 1.0) * (x - 3.0) + cos(x);

        if (f > m) {
            m = f;           // Record new global maximum
            x_at_max = x;     // Record location
        }
    }

    cout << "=====\n";
    cout << "  NUMERICAL FUNCTION MAXIMIZATION RESULT\n";
    cout << "=====\n";
    cout << "Maximum Value m = " << m << "\n";
    cout << "Occurs at x      = " << x_at_max << " radians\n";
    cout << "Domain Limit    = " << x_end << " radians\n";

    return 0;
}
```

Engineering Analysis:

- The term $(x - 3.0)$ scales linearly with x , while $\sin(x^2 - 1.0)$ oscillates between -1 and $+1$.
- Near the end of the domain ($x \approx 10\pi \approx 31.416$), the amplitude multiplier $(x - 3.0)$ reaches approximately $31.416 - 3.0 = 28.4$.
- When $\sin(x^2 - 1.0) \approx +1$, the function peak approaches $m \approx 28.4 + 1.0 \approx \mathbf{29.4}$.
- The numerical discretization ($dx = 0.01$) samples over 3,142 grid points, capturing this global peak with high precision.